Examples of Generating Options Patterns

Conversation Log | June 27, 2026

USER

Give examples for each of the patterns in Generating options category

CLAUDE

Here's one for each of the three patterns under Generating Options — Brainstorming, Tree of Thoughts, and Analogical Reasoning. They're related but distinct in how the options get generated and judged.

Brainstorming

Ask for many candidate ideas up front, with judgment deliberately deferred — the goal is breadth, not depth on any one idea yet.

> *"I want to add a new output mode to TextFinder1 beyond plain stdout — give me 6-8 different ideas for how results could be presented (formats, destinations, interactivity, whatever comes to mind). Don't evaluate them yet, just generate a wide list."*

The model might list: JSON output for piping into other tools, a TUI with scrollable results, colorized terminal highlighting of matches, CSV export, a --watch mode that re-runs on file changes, grouping by directory instead of file, an interactive mode that lets you jump to a match in $EDITOR, and a summary-only mode showing match counts per file.

Why it's distinct: no structure is imposed on the ideas — they don't need to be mutually exclusive, ranked, or even fully baked. The point is surfacing options you wouldn't have thought of, before any cost/benefit thinking narrows the field.

Tree of Thoughts

Generate multiple reasoning paths (not just options), explore each a few steps deep, and prune branches that dead-end — useful when the "right answer" depends on reasoning that could go several plausible directions.

> *"TextFinder1 is slow on a 50,000-file repo. Explore three different optimization branches in parallel: (1) parallelizing file reads across threads, (2) skipping files via a fast pre-filter before full scanning, (3) memory-mapping files instead of buffered reads. For each branch, reason two steps ahead — what's the next bottleneck after this fix — and tell me which branch hits a dead end soonest."*

The model might trace: thread-parallelism helps until you're I/O-bound rather than CPU-bound, at which point more threads stop helping and you've added complexity for nothing. The pre-filter branch reduces files scanned but the filter itself still touches every file once, so its ceiling is bounded by directory traversal speed. Memory-mapping reduces syscall overhead per file but doesn't help if the real cost is regex matching itself.

Why it's distinct from Brainstorming: each branch isn't just an idea, it's a reasoning chain you follow forward to see where it leads — and branches get cut not because they're “bad ideas” but because tracing them reveals a ceiling or contradiction.

Analogical Reasoning

Map the current problem onto a structurally similar one the model (or you) already understands well, and borrow its solution shape.

> *“TextFinder1 needs to support multiple simultaneous search patterns with different match rules per pattern — similar to how a build system runs multiple independent rules against a file set. How would a build system's rule-dispatch model translate to a multi-pattern matcher?”*

The model might map: a build system associates each rule with a target pattern and an action; TextFinder1 could associate each search pattern with a MatchMode and an output formatter the same way, dispatching each file through every active rule independently rather than hard-coding a single global pattern. The “dependency graph” part of build systems doesn't transfer (your searches don't depend on each other), but the “list of independent rule objects, each evaluated against the same input” structure does.

Why it's distinct: it doesn't generate options by listing alternatives or branching reasoning — it generates one option by importing the shape of a solution from a different, more familiar domain, then checking which parts of the analogy hold and which break down.

Together, the three give you different levers: Brainstorming widens the set of ideas, Tree of Thoughts widens the reasoning paths toward one idea, and Analogical Reasoning imports a ready-made structure from somewhere else entirely.